



# INDIA.RUNS

Build what next India runs on

## TRACK 1 · THE DATA & AI CHALLENGE

# Intelligent Candidate Discovery

### TEAM

dolphin

### TEAM LEADER

Viraj Phad

### TEAM MEMBERS

Anushka Kulkarni  
Chirag Pednekar

**PROBLEM** Rank the 100 best-fit candidates from a 100,000-profile pool against a nuanced JD — seeing past keywords to true fit.

## SOLUTION OVERVIEW

# An AI recruiter that reads the whole story

We rank candidates by INTERNAL COHERENCE + JD-alignment across every field and whether the person is actually available — not by counting keywords. The same system re-derives its target for any JD and any pool.

## ● Coherence over keywords

The dataset is adversarial — keyword/embedding matching ranks scrambled 'stuffers' #1. We demote them by checking the fields agree.

## ● Career ≠ keywords

A trust layer separates an IR career from IR buzzwords — demoting CV/services/'lightweight' generalists who only list the skills.

## ● Behavioral availability

Down-weights perfect-on-paper ghosts who haven't logged in for months or ignore recruiters — they aren't actually hireable.

## ● Fast, CPU-only, general

~35s over 100K on CPU, no per-candidate LLM calls.  
Nothing hardcoded to this JD — it productizes to any role.

## JD UNDERSTANDING &amp; CANDIDATE EVALUATION

# From a rambling JD to a precise target

## ● Requirements (auto-parsed)

A faceted JobSpec extracted from the JD text: production retrieval / ranking / embeddings / vector-DBs, 5–9 yrs, product (not services) companies, Pune/Noida + relocate, rigorous eval (NDCG/MRR).

## ● Disqualifiers it catches

Services-only careers, pure-research, LangChain-only/<12mo 'AI', computer-vision / speech without NLP-IR, and title-chasers — all called out in the JD, all parsed into the spec.

## ● Signals that matter most

Cross-field coherence; career-IR-focus (do the role descriptions describe ranking work?); corroborated skills (endorsements × duration × assessments); behavioral availability.

**Beyond keywords:** the master signal is internal coherence + JD-alignment across ALL fields at once. Real fits tell one consistent story; traps are stitched from mismatched parts.

## RANKING METHODOLOGY

# Retrieve → score → rank, explainably

```
score = alignment (lexical + semantic) × coherence × trust  
(career#keywords) × availability - honeypot sink
```

## ● Models & algorithms

MiniLM-L6 embeddings (computed offline), numpy cosine + percentile, and deterministic feature scoring. No hosted-LLM call per candidate — it must scale to millions.

## ● Combining the signals

Additive core (alignment + coherence) × multiplicative dealbreakers (availability, trust), minus a near-zero honeypot sink — tier-like by design and robust (top-10 invariant to ±20% weight change).

## EXPLAINABILITY &amp; DATA VALIDATION

# Honest reasons, no hallucinations

## ● Feature-grounded reasons

Each candidate's reasoning is generated from the real features that drove its score — title, years, a named employer, signal values — with honest concerns, and tone that scales with rank.

## ● No hallucination

Only self-consistent fields are cited (an employer is named only when the profile is coherent). Verified: every cited employer actually exists in that candidate's history.

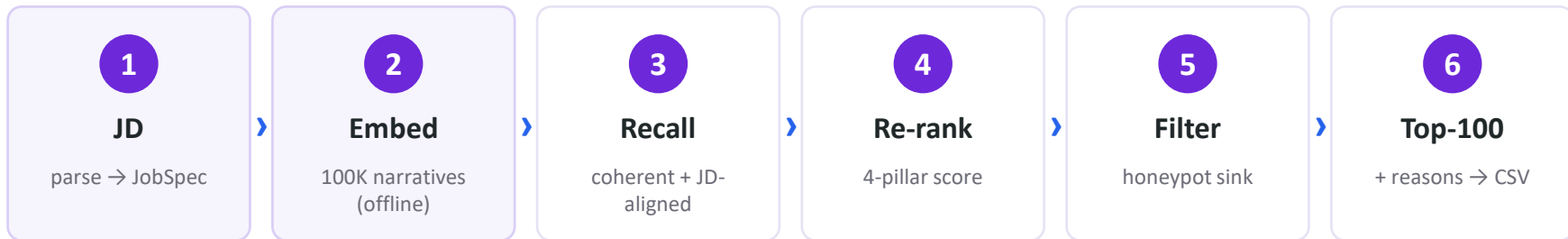
## ● Suspicious profiles

Honeypots (impossible dates / tenure / 'expert' + 0 months) hit a logic sink → 0% in the top-100. Scrambled stuffers and CV/services generalists are demoted by coherence + trust.

**0% honeypots** in the top-100 (disqualification threshold is 10%) · cited employers verified against the raw profile.

## END-TO-END WORKFLOW

# From JD input to ranked output



**Offline eval harness** grades every change on NDCG@10/@50, MAP, P@10 against 68 hand-read labels — our substitute for the missing leaderboard (3 submissions, no live feedback).

## SYSTEM ARCHITECTURE

# Two phases: precompute offline, rank fast

## OFFLINE · UNTIMED

JD → JobSpec (faceted)

100K narratives → MiniLM  
embeddings

save vectors → artifacts/\*.npy

## RANKING STEP · CPU · numpy only · &lt; 5 min · no network

load vectors + features

score: align × coherence × trust ×  
avail – honeypot

top-100 + reasons → submission.csv

*Retrieve → re-rank → filter → explain → evaluate — standard production IR shape; torch never imported at rank time.*

## RESULTS &amp; PERFORMANCE

# Strong, honest, and within budget

**1.00****NDCG@10**

top-10 hand-verified

**~0.996****composite (offline)**

gold-anchored

**0%****honeypots in top-100**

DQ line is 10%

**~35s****for 100K, CPU-only**

budget is 5 min

**Constraints met:** CPU-only, no network at rank time, numpy-only rank step, ~400 MB peak — well under the 5 min / 16 GB / no-GPU limits, and reproducible in a sandboxed container.

**Honest note:** the composite is anchored on 68 hand-read relevance labels (top-35 well covered); deeper ranks carry some weaker-label noise — our best offline estimate, not a guarantee.



## TECHNOLOGIES USED

# Chosen for the 5-minute, CPU-only bar

**Python + numpy / scipy / pandas**

The entire ranking step — deliberately minimal so it reproduces in the Stage-3 sandbox.

**sentence-transformers + PyTorch (CPU)**

MiniLM-L6 embeddings, computed offline. Chosen over bge-small for speed (78/s vs 11/s).

**scikit-learn / SciPy**

Percentile normalization, silver labels, and NDCG/MAP metrics in the eval harness.

**PyYAML**

The JobSpec artifact — 'parse the JD once', then consumed by the ranker.

**Google Colab**

Hosted sandbox demo that runs the pipeline on a sample, end-to-end on CPU.

**Git**

Real iterative history — every design choice backed by a measured experiment.

## SUBMISSION ASSETS

# Everything, reproducible

**Ranked output**

submission.csv — top-100, ranked, each with a 1-line reason

**GitHub repository**

[github.com/candyflipgit/redrob-candidate-ranker](https://github.com/candyflipgit/redrob-candidate-ranker)

**Live sandbox (Colab)**

runs the full pipeline on a sample, end-to-end on CPU, [here](#)

**Blueprint**

README methodology + <200-word summary + 16-commit history



# *INDIA.RUNS*

Build what next India runs on

# THANK YOU